

Types and arithmetic expressions

CSC103 Programming Fundamentals / Lecture 05

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Convert 3672 seconds to hours, minutes and seconds using / and %.

Find complete hours with seconds / 3600.

Learning objectives

- Choose suitable primitive types
- Evaluate precedence and integer division
- Use casts and conversions deliberately

CLO-1

Primitive data types

Division and remainder

Precedence and grouping

Conversions and numeric limits

Primitive data types

- Integer types: byte, short, int and long.
- Floating-point types: float and double. Other primitives: char and boolean.
- A String is a reference type.

Example

```
int count = 20;  
long population = 8000000000L;  
double average = 72.5;  
boolean passed = true;
```

Division and remainder

- Integer division truncates toward zero.
- The % operator returns a remainder. Its sign follows the dividend.
- At least one floating-point operand gives floating-point division.

Example

```
7 / 2      // 3
7 / 2.0    // 3.5
7 % 2      // 1
-7 / 2     // -3
```

Precedence and grouping

- Multiplication, division and remainder bind before addition and subtraction.
- Operators at these arithmetic levels associate left to right.
- Parentheses make the intended calculation explicit.

Example

```
2 + 3 * 4      // 14
(2 + 3) * 4    // 20
20 / 5 * 2     // 8
```

Conversions and numeric limits

- Widening conversions often happen automatically.
- A narrowing cast can discard a fraction or lose range.
- Floating-point values approximate many decimal fractions.

Example

```
double x = 7;  
int y = (int) 7.9; // 7  
double mean = (double) 7 / 2;
```

Quotient and remainder decompose a quantity

- Whole-unit conversion uses integer division to count complete larger units.
- Remainder keeps the part that has not yet been converted.
- Convert the remainder again when more than two units are needed.

Example

3672 seconds

1 complete hour leaves 72 seconds

1 complete minute leaves 12 seconds

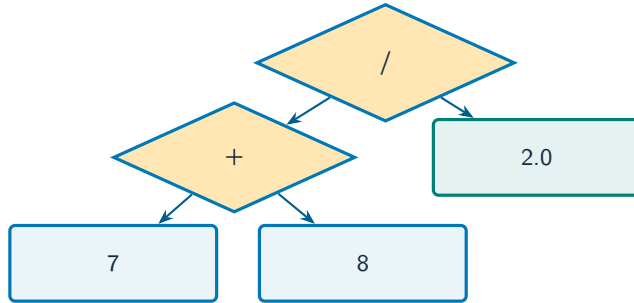
Casting changes a value at a specific point

- A cast affects the expression at the location where it occurs.
- Casting an already truncated quotient cannot restore its discarded fraction.
- Promote an operand before division when a fractional result is required.

Example

```
(double) (7 / 2) gives 3.0  
(double) 7 / 2 gives 3.5
```

Arithmetic follows the expression tree



What to notice

The denominator 2.0 makes the final division floating-point.

Key distinctions

Expression	Operand arithmetic	Result
7 / 2	Integer division	3
7 / 2.0	Floating-point division	3.5
7 % 2	Integer remainder	1
(int) 7.9	Narrowing conversion	7
(double) (7 / 2)	Cast after division	3.0

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int total = 145;
int count = 2;
double wrong = total / count;
double correct = (double) total / count;
System.out.println(wrong);
System.out.println(correct);
```

First example: result

72.0

72.5

The first expression performs integer division before assignment.

The cast changes an operand before division.

Precedence and grouping

Conversions and numeric limits

Guided practice

Convert 3672 seconds to hours, minutes and seconds using / and %.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Find complete hours with $\text{seconds} / 3600$.
- Keep $\text{seconds} \% 3600$ for the part after complete hours.
- Divide that remainder by 60, then use $\% 60$ for remaining seconds.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution05 {  
    public static void main(String[] args) throws Exception {  
        int totalSeconds = 3672;  
        int hours = totalSeconds / 3600;  
        int remainder = totalSeconds % 3600;  
        int minutes = remainder / 60;  
        int seconds = remainder % 60;  
        System.out.println(hours + ":" + minutes + ":" + seconds);  
    }  
}
```


Execution trace

Variable	Calculation	Value
hours	$3672 / 3600$	1
remainder	$3672 \% 3600$	72
minutes	$72 / 60$	1
seconds	$72 \% 60$	12

Follow the changing state in execution order.

Solution result

1:1:12

Why this result follows
Divide that remainder by 60, then
use % 60 for remaining seconds.

A common incorrect approach

```
double average = (double) (7 / 2);
```

Example

The cast is too late. Use `(double) 7 / 2` to obtain 3.5.

Boundary and failure checks

Check the stated input assumptions.

Write a temperature converter. Test 0 and 100 Celsius.

Explain why $9 / 5$ gives the wrong conversion factor.

hours = $3672 / 3600 = 1$

minutes = $(3672 \% 3600) / 60 = 1$

seconds = $3672 \% 60 = 12$

Check your understanding

Evaluate $10 - 3 * 2$ and $(10 - 3) * 2$. What does `(int) -2.9` produce?

Write a prediction and explain the rule that supports it.

Answer and explanation

4, 14 and -2. The cast truncates toward zero.

The cast is too late. Use (double) 7 / 2 to obtain 3.5.

Compare cases and predict outcomes

Expression	Result	Reason
$(7 + 8) / 2$	7	Integer quotient
$(7 + 8) / 2.0$	7.5	Floating-point divisor
$(\text{double}) ((7 + 8) / 2)$	7.0	The fraction was already lost

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Convert 5,439 seconds to hours, minutes and seconds. Explain why quotient and remainder are both needed.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int seconds = 5439;
int hours = seconds / 3600;
int remainder = seconds % 3600;
int minutes = remainder / 60;
int leftover = remainder % 60;
System.out.println(hours + ":" + minutes + ":" + leftover);
```

- The first quotient gives 1 complete hour; 1,839 seconds remain.
- The next quotient gives 30 minutes; the remainder is 39 seconds.
- Output: 1:30:39.

Division and compound assignment

- Integer division discards the fractional part; a floating-point operand changes the arithmetic.
- A compound assignment includes conversion back to the left variable type.
- Therefore $x /= 3.0$ for an int x is not equivalent to the uncompileable $x = x / 3.0$.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB_Source_Map.md](#).

Worked problem: Division and compound assignment

Task

Compare $5/2$, $5.0/2$ and `int x=10` followed by `x/=3.0`. Explain each result.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture05 {  
    public static void main(String[] args) throws Exception {  
        System.out.println(5 / 2);  
        System.out.println(5.0 / 2);  
        int x = 10;  
        x /= 3.0;  
        System.out.println(x);  
    }  
}
```

Worked trace and expected result

Expression	Arithmetic before storage	Result
5 / 2	Integer	2
5.0 / 2	double	2.5
x /= 3.0, x initially 10	double then int conversion	3

Expected console output

2
2.5
3

Extend the ISB example

Practice

Rewrite `x /= 3.0` with an explicit assignment that compiles and preserves the demonstrated conversion.

Explain your reasoning

`x = (int) (x / 3.0);` The cast occurs after division.

Lecture summary

- suitable primitive types
- precedence and integer division
- casts and conversions deliberately
- Find complete hours with $\text{seconds} / 3600$.
- Keep $\text{seconds} \% 3600$ for the part after complete hours.
- Divide that remainder by 60, then use $\% 60$ for remaining seconds.

After-class practice

Write a temperature converter. Test 0 and 100 Celsius.
Explain why $9 / 5$ gives the wrong conversion factor.

Syllabus reading

Liang Ch 2

Next lecture: Updates and formatted output